1 Project Definition Statement (PDS)

> *Versione estesa del POS, destinata al planning team. Stesse cinque sezioni, con livello di dettaglio maggiore.*

Campo	Valore
Nome progetto	DroneFlow — Servizio di Consegna Pacchi con Droni
Project No.	SAP-03
Project Manager	Ghignatti Nicolò
Versione	1.0 — Aprile 2026

Tabella 1.1

1.1 1. Problema / Opportunità

1.1.1 Contesto

Il mercato delle consegne dell'ultimo miglio è caratterizzato da una domanda crescente di rapidità, flessibilità e trasparenza. Gli utenti che necessitano di spedire un pacco si trovano di fronte a servizi che impongono finestre di consegna ampie e poco prevedibili, senza alcuna visibilità sul percorso del pacco una volta che questo è stato affidato al corriere.

1.1.2 Il bisogno dell'utente

Un utente vuole poter:

- Richiedere il ritiro di un pacco **da un indirizzo specifico verso una destinazione**, indicando peso, momento preferito di ritiro e tempo massimo entro cui completare la

consegna.

- Sapere immediatamente se la sua richiesta è accettabile (il pacco è trasportabile, il servizio è disponibile in quella finestra).
- Seguire **in tempo reale** il percorso del drone che trasporta il suo pacco.
- Ricevere conferma dell'avvenuta consegna.

1.1.3 Il gap attuale

I servizi di corriere tradizionali non offrono:

- Garanzia di rispetto di una finestra temporale stretta (es. "consegna entro 30 minuti").
- Tracciabilità in tempo reale della posizione fisica del pacco durante il trasporto.
- Automazione completa dalla ricezione della richiesta all'assegnazione del vettore.

DroneFlow risolve questi tre gap con un servizio completamente automatizzato, basato su droni.

1.2 2. Obiettivo del progetto

Realizzare il prototipo funzionante di **DroneFlow**: un servizio online che consenta a un utente di richiedere la consegna di un pacco tramite drone, specificando origine, destinazione, peso, orario e tempo massimo di consegna, e di seguirne lo stato in tempo reale fino alla conferma di avvenuta consegna.

Il prototipo deve dimostrare che l'intero ciclo — dalla richiesta utente all'assegnazione del drone, fino alla conferma finale — può avvenire in modo **completamente automatico**, con gestione corretta anche dei casi di fallimento (richiesta non soddisfabile, drone non disponibile).

1.3 3. Obiettivi specifici

1.3.1 Obiettivo 1 — Ricezione e validazione delle richieste

Il sistema deve accettare richieste di consegna. Prima di procedere, ogni richiesta deve essere verificata automaticamente: il pacco deve avere un peso compatibile con la capacità dei droni, gli indirizzi devono essere specificati, la finestra temporale deve essere coerente. Le richieste non valide devono essere rifiutate con un messaggio chiaro per l'utente.

1.3.2 Obiettivo 2 — Pianificazione e assegnazione automatica

Ogni richiesta valida deve essere automaticamente pianificata: il sistema seleziona un drone disponibile, gli assegna la missione di consegna e lo invia verso l'indirizzo di ritiro. Questo processo non richiede alcun intervento manuale da parte degli operatori del servizio.

1.3.3 Obiettivo 3 — Tracciabilità in tempo reale

Durante tutta la durata della consegna, l'utente deve poter consultare lo stato del proprio pacco: il sistema deve esporre aggiornamenti periodici sulla posizione del drone e fornire una stima aggiornata del tempo rimanente.

1.3.4 Obiettivo 4 — Gestione dei fallimenti e annullamento ordinato

Se in qualsiasi momento del processo si verifica un problema (drone che diventa non disponibile dopo l'assegnazione, errore di comunicazione, ecc.), il sistema deve annullare la consegna in modo ordinato: ogni operazione parzialmente completata viene disfatta, le risorse vengono liberate e l'utente viene informato del motivo dell'annullamento.

1.3.5 Obiettivo 5 — Storico delle consegne

Il sistema deve conservare uno storico permanente e immutabile di ogni consegna: ogni cambio di stato (drone assegnato, partito, arrivato, tornato) è registrato con timestamp. Questo storico è consultabile sia dall'utente per le proprie spedizioni, sia dagli operatori per controllo qualità e auditing.

1.3.6 Obiettivo 6 — Scalabilità del servizio

L'architettura del sistema deve consentire di gestire un numero crescente di richieste simultanee senza degrado delle prestazioni. Ogni componente deve poter essere scalato indipend-

entemente dagli altri. *(Questo è un vincolo tecnico che guida le scelte architetturali, non un requisito funzionale diretto per l'utente.)*

1.4 4. Criteri di successo

ID	Criterio	Come si verifica	Target
CS1	Richiesta valida accettata e drone assegnato	Demo: POST richiesta → conferma con tracking ID	100%
CS2	Richiesta non valida rifiutata con messaggio	Demo: richiesta con peso negativo → risposta con errore leggibile	100%
CS3	Stato della consegna consultabile in ogni momento	API di tracking risponde con stato aggiornato durante tutta la consegna	100%
CS4	Storico eventi del drone coerente con l'evoluzione reale	Tutti gli eventi (partenza, arrivo, rientro) presenti e ordinati	100%
CS5	Annullamento ordinato in caso di fallimento	Demo con fallimento simulato → nessun dato bloccato, utente informato	100%
CS6	Documentazione PM completa e coerente con la proposal	7 documenti presenti, in italiano, coerenti tra loro	✓

Tabella 1.2

1.5 5. Assunzioni, Rischi e Ostacoli

1.5.1 Assunzioni

- Il peso massimo trasportabile da un drone è noto e configurabile (parametro di sistema).

- L'area di operatività del servizio è delimitata (non si gestiscono consegne intercontinentali o in zone non coperte).
- Nel prototipo i droni sono simulati software; la posizione è approssimata.
- Non sono richiesti pagamenti, autenticazione avanzata o integrazione con sistemi esterni nella versione prototipale.

1.5.2 Rischi dettagliati

R1 — Volume di richieste simultanee superiore alla flotta disponibile

- **Descrizione**: In un picco di domanda, tutti i droni potrebbero essere in missione.
- **Impatto**: Utenti che ricevono un rifiuto per indisponibilità.
- **Mitigazione**: Il sistema deve comunicare chiaramente l'impossibilità di servire la richiesta in quel momento, senza lasciare la richiesta in stato indefinito.

R2 — Perdita di comunicazione con un drone in volo

- **Descrizione**: Il drone non aggiorna più la propria posizione durante una consegna.
- **Impatto**: L'utente vede uno stato bloccato; la risorsa risulta occupata.
- **Mitigazione**: Il sistema registra lo storico degli eventi in modo immutabile; è possibile ricostruire l'ultimo stato noto e attivare procedure di recupero.

R3 — Dati di richiesta errati o incompleti

- **Descrizione**: L'utente inserisce indirizzi non validi o pesi fuori specifica.
- **Impatto**: La consegna non può partire.
- **Mitigazione**: Validazione automatica prima della presa in carico; risposta immediata con indicazione dell'errore.

R4 — Scope creep (aggiunta di funzionalità non previste)

- **Descrizione**: Durante lo sviluppo si tende ad aggiungere features non pianificate (pagamenti, gestione flotta avanzata, notifiche push, ecc.).
- **Impatto**: Ritardi, documentazione non coerente.

- **Mitigazione**: Qualsiasi aggiunta deve essere valutata rispetto al POS prima di essere intrapresa.

1.5.3 Vincoli tecnici (non obiettivi del progetto, ma vincoli di realizzazione)

- Il sistema è realizzato come insieme di servizi indipendenti che comunicano in modo asincrono, per garantire scalabilità e isolamento dei guasti.
- I dati di ogni consegna sono persistiti su database non relazionale (MongoDB).
- La comunicazione tra i componenti interni avviene tramite message broker (RabbitMQ).
- Il linguaggio di implementazione è Rust (edizione 2024).
- La coordinazione di operazioni distribuite in caso di fallimento segue il pattern SAGA.
- Lo storico degli eventi del drone è implementato tramite Event Sourcing.